DZIEN 4 — DICTIONARY, ROZBUDOWANE STATE MACHINE, REFRAMEWORK

Modul 1 — Dictionary

CZYM JEST DICTIONARY?

Slownik (Dictionary) to kolekcja par **klucz-wartosc**. Kazdy klucz jest unikalny — nie moga istniec dwa elementy z tym samym kluczem. Najprosciej myslec o nim jak o encyklopedii: szukasz hasla (klucz) i dostajesz definicje (wartosc).

> **Przypomnienie z Dnia 1-2:** Na Dniu 1 omowilismy podstawy kolekcji (Array, List, Dictionary). Teraz pokazemy praktyczne zastosowanie Dictionary w realnym scenariuszu — mapowanie kursow walut.

W UiPath najczesciej spotykane typy:

- `Dictionary(Of String, String)` — klucz i wartosc to tekst (np. kod waluty → kurs)
- `Dictionary(Of String, Object)` — klucz to tekst, wartosc to dowolny typ (tak dziala Config w REFramework)
- `Dictionary(Of String, Int32)` — klucz to tekst, wartosc to liczba (np. licznik kategorii)

DICTIONARY VS DATATABLE — KIEDY CO WYBRAC?

Kryterium	Dictionary	DataTable
Struktura	Klucz → Wartosc (1:1)	Wiersze x Kolumny (tabela)
Wyszukiwanie	Blyskawiczne po kluczu (O(1))	Wymaga iteracji po wierszach (O(n)) lub LINQ
Kiedy uzyc	Mapowanie (kod → nazwa), konfiguracja, cache	Dane tabelaryczne, raporty, import/eksport Excel
Przyklad	Kod waluty → kurs	Lista transakcji z wieloma kolumnami
Sortowanie	Brak gwarancji kolejnosci	Zachowuje kolejnosc wierszy

ZLOZONOSC OBLICZENIOWA — DLACZEGO DICTIONARY JEST SZYBSZY?

Gdy masz 10 000 walut w DataTable i szukasz kursu EUR, musisz przejsc potencjalnie przez wszystkie 10 000 wierszy (w najgorszym przypadku). To zajmuje czas proporcjonalny do rozmiaru tabeli.

Dictionary uzywa techniki zwanej "haszowaniem" — zamienia klucz na liczbe i od razu wie, w ktorym "schowku" szukac. Niezaleznie od tego czy masz 100 czy 10 000 elementow, czas wyszukiwania jest staly (**O(1)**).

> **W praktyce:** jesli musisz wielokrotnie wyszukiwac wartosc po kluczu (np. kurs waluty, tlumaczenie kodu na nazwe), Dictionary bedzie **znacznie szybszy** niz iterowanie po DataTable.

INICJALIZACJA

```
' W Assign:
dict_waluty = New Dictionary(Of String, String)
```

> **Wazne:** Oprocz stworzenia zmiennej w Data Manager i ustawienia typu, trzeba jeszcze napisac komende `New Dictionary(Of String, String)` w Assign — inaczej zmienna bedzie `Nothing` i kazda operacja rzuci `NullReferenceException`!

Operacje na Dictionary

Operacja	Wyrazenie VB.NET	Opis
Dodaj/Ustaw wartosc	`dict("klucz") = "wartosc"`	Dodaje lub nadpisuje
Pobierz wartosc	`dict("klucz")`	Rzuca wyjatek jesli klucz nie istnieje
Sprawdz klucz	`dict.ContainsKey("klucz")`	Zwraca Boolean
Liczba elementow	`dict.Count`	Int32
Usun element	`dict.Remove("klucz")`	Zwraca Boolean

TYPOWE PULAPKI PRZY PRACY Z DICTIONARY

1. KeyNotFoundException

Proba pobrania wartosci kluczem, ktory nie istnieje:

```
' ZLE — rzuci wyjatek jesli "GBP" nie istnieje:
str_kurs = dict_waluty("GBP")

' DOBRE — najpierw sprawdz:
If dict_waluty.ContainsKey("GBP") Then
    str_kurs = dict_waluty("GBP")
End If
```

2. Case-sensitivity kluczy

Domyslnie Dictionary rozroznia wielkosc liter:

```
dict_waluty("EUR") = "4.55"
dict_waluty("eur")  ' → KeyNotFoundException! "eur" ≠ "EUR"
```

> **Rozwiazanie:** przy dodawaniu i pobieraniu zawsze stosuj `.ToUpper` lub `.ToLower` na kluczu.

3. NullReferenceException

Dictionary nie zostalo zainicjalizowane:

```
' Stworzenie zmiennej w Data Manager to za malo!
' Trzeba jeszcze:
dict_waluty = New Dictionary(Of String, String)
```

Cwiczenia — Modul 1

Cwiczenie — Kursy walut NBP → Dictionary

Cel: Otworz strone NBP z kursami walut, pobierz tabele (Table Extraction), zapisz do Excela, przypisz do Dictionary i wyswietl kurs Euro.

CZEGO SIE NAUCZYSZ

- `Extract Table Data` — scrapowanie tabel ze stron www
- Inicjalizacja Dictionary — MUSI byc `New Dictionary(Of String, String)` w Assign
- `dict(klucz) = wartosc` — jednoczesnie dodaje i nadpisuje
- Dostep do wartosci: `dict_waluty("1 EUR").ToString`
- Klasyczny pattern mapowania danych do Dictionary: jedna kolumna jako klucze, druga jako wartosci

Modul 2 — Rozbudowane State Machine

BUSINESSRULEEXCEPTION VS SYSTEMEXCEPTION — KLUCZOWE ROZROZNIENIE

W automatyzacji procesow wyrozniamy dwa fundamentalnie rozne typy bledow:

Cecha	BusinessRuleException (BE)	System Exception (SE)
Definicja	Blad wynikajacy z danych lub regul biznesowych	Blad techniczny/systemowy
Przyklady	Brak NIPu w formularzu, kwota ujemna, niepoprawny email, dane nienumeryczne	Strona sie nie zaladowala, aplikacja przestala odpowiadac, timeout, brak internetu
Czy wina robota?	Nie — dane wejsciowe sa zle	Nie — srodowisko jest niestabilne
Czy ponawiac?	**NIE** — ponawianie z tymi samymi zlymi danymi da ten sam wynik	**TAK** — ponowna proba moze sie udac (np. strona sie zaladuje)
Reakcja w SM	Pomin transakcje (Failed + Business) → nastepna	**Restart** — zamknij aplikacje, otworz ponownie, sprobuj jeszcze raz
W UiPath	`Throw New BusinessRuleException("opis")`	Nieobsluzony Exception (np. SelectorNotFoundException, TimeoutException)

Dlaczego to rozroznienie jest wazne?

Wyobraz sobie, ze robot przetwarza faktury. Faktura #123 ma literowke w NIPie (BE) — ponawianie 10 razy nic nie da. Ale jesli strona podatki.gov.pl nie odpowiada (SE) — po 30 sekundach moze zadzialac.

> **W REFramework to rozroznienie jest wbudowane:**
> BE → transakcja oznaczona jako Failed (Business), robot przechodzi do nastepnej
> SE → robot restartuje aplikacje (wraca do stanu Init) i ponawia probe

STANY PROCESU W ROZBUDOWANEJ STATE MACHINE

Stan	Opis	Transitions
State - Init	Inicjalizacja: zbierz config, otworz aplikacje, pobierz dane	Sukces → Pobierz transakcje; SE → Final State
State - Pobierz jedna transakcje	Sprawdz czy sa dane, pobierz wiersz z DataTable	Sukces → Procesowanie; Koniec → Final State
State - Procesowanie jednej transakcji	Wpisz dane do Robotivity, obsluz Business/System Exception	Sukces → Pobierz transakcje; SE → Init (restart!)
Final State	Loguj status, zamknij aplikacje	— (koniec)

Diagram przeplywu stanow

```
                    +-------------------+
                    |   State - Init    |
                    |  (konfiguracja,   |
                    |  otwarcie aplik.) |
                    +--------+----------+
                   Sukces |     | SE (blad krytyczny)
          +----------------+   |
          | State - Pobierz |   |
          |    transakcje   |   |
          +-------+---------+   |
         Sukces |  | Koniec     |
   +--------------+  |          |
   |   State -    |  |          |
   | Procesowanie |--SE----> State - Init (restart!)
   +------+-------+             |
     Sukces |                   |
   (powrot do Pobierz)    +-------------+
                          | Final State |
                          |  (zamknij)  |
                          +-------------+
```

KLUCZOWE ZMIENNE

Nazwa	Typ	Opis
TransactionNumber	Int32 (default=1)	Numer biezacego wiersza
dict_Config	Dictionary(Of String, String)	Ustawienia (FilePath, SheetName)
dt_transakcje	DataTable	Dane z Transactions.xlsx
row_TransactionItem	DataRow	Biezacy wiersz (transakcja)
SystemException	Exception	Wyjatek systemowy (jesli wystapi)
BusinessRuleException	BusinessRuleException	Wyjatek biznesowy

LOGIKA POSZCZEGOLNYCH STANOW

State - Init

1. **If** — `dict_Config is Nothing` → pierwsze uruchomienie:
   - Zbierz ustawienia do Dictionary (FilePath, SheetName)
   - Kill Process `excel`

- Read Range → `dt_transakcje`

2. Kill Process `robotivity` (wyczysc srodowisko)
3. Otworz Robotivity + zaloguj sie (z Retry Scope!)
4. **Try Catch** wokol wszystkiego — jesli Exception → `SystemException = exception`
5. Transitions:
   - `SystemException IsNot Nothing` → Final State (blad krytyczny)
   - Else → State - Pobierz transakcje

State - Pobierz jedna transakcje

1. Sprawdz `TransactionNumber <= dt_transakcje.Rows.Count`
2. Jesli tak → `row_TransactionItem = dt_transakcje.Rows(TransactionNumber - 1)`
3. Jesli nie → `row_TransactionItem = Nothing`
4. Transitions:
   - `row_TransactionItem Is Nothing` ("Koniec") → Final State
   - Else ("Sukces") → State - Procesowanie

State - Procesowanie

1. Wyczysc `BusinessRuleException = Nothing` (wazne, aby nie wejsc w nieskonczona petle!)
2. Wyciagnij CashIn, OnUsCheck, NotOnUsCheck z `row_TransactionItem`
3. Sprawdz `.IsNumeric` — jesli nie → `BusinessRuleException = New BusinessRuleException("Dane nie sa numeryczne")`
4. Jesli numeryczne → Type Into w Robotivity + Click Accept
5. `TransactionNumber = TransactionNumber + 1`
6. Transitions:
   - `SystemException IsNot Nothing` ("SE") → State - Init (restart!)
   - Else ("Sukces") → State - Pobierz transakcje

Final State

- If `SystemException Is Nothing` → "Robot wykonal prace" (Info)
- Else → "Blad pracy robota: " + `SystemException.Message` (Error)
- Kill Process `robotivity`

Cwiczenia — Modul 2

Cwiczenie — Rozbudowane State Machine

Cel: Zbuduj State Machine z pelna obsluga wyjatkow — wzorzec zblizony do REFramework. Robot przetwarza transakcje z Excela w Robotivity, rozrozniajac bledy biznesowe (dane nienumeryczne) od systemowych (awaria aplikacji).

CZEGO SIE NAUCZYSZ

- Dictionary jako Config — wzorzec REFramework (Key-Value settings)
- `If dict_Config is Nothing` — inicjalizacja tylko raz (przy restarcie nie powtarza)
- Retry Scope — automatyczne ponawianie logowania (2 proby, 3s odstep)
- BusinessRuleException vs SystemException — rozne typy bledow, rozne reakcje
- Czyszczenie BusinessRuleException na poczatku procesowania — zapobiega nieskonczonej petli
- Transition "SE" z Procesowania → Init — restart aplikacji po bledzie systemowym
- TransactionNumber — reczny licznik wierszy (zamiast Get Transaction Item z Orchestratora)

Wazne: To jest bezposredni wstep do REFramework! Zauwazyj analogie: Init ↔ InitAllApplications, Pobierz transakcje ↔ GetTransactionData, Procesowanie ↔ Process, Final State ↔ CloseAllApplications.

Modul 3 — REFramework: wprowadzenie

DLACZEGO REFRAMEWORK? JAKIE PROBLEMY ROZWIAZUJE?

Wyobraz sobie, ze piszesz robota, ktory przetwarza 500 faktur dziennie. Bez frameworka musisz sam zaprogramowac:

- Co sie stanie, gdy strona sie zawiesi w trakcie przetwarzania faktury #237?
- Jak zrestartowac aplikacje i wrocic do przetwarzania?
- Ile razy ponawiac probe przed oznaczeniem transakcji jako "Failed"?
- Gdzie trzymac konfiguracje (URL, hasla, timeouty)?
- Jak logowac postep i bledy?

Kazdy programista rozwiaze to inaczej — co prowadzi do chaosu w utrzymaniu. REFramework standaryzuje te rozwiazania.

CO REFRAMEWORK ZAPEWNIA "OUT OF THE BOX"

1. **Automatyczny retry** — po System Exception robot restartuje aplikacje i probuje ponownie (konfigurowalne MaxRetryNumber)
2. **Rozroznienie bledow** — Business Exception (pomin transakcje) vs System Exception (restart)
3. **Centralna konfiguracja** — Config.xlsx zamiast hardkodowanych wartosci
4. **Logging** — automatyczne logowanie statusow transakcji
5. **Screenshots przy bledach** — TakeScreenshot.xaml robi zrzut ekranu przy wyjatku
6. **Separacja kodu** — Framework (nie ruszaj) vs Process.xaml (twoja logika)

STRUKTURA FOLDEROW REFRAMEWORK

Folder/Plik	Opis
`Main.xaml`	Glowny State Machine (Init → Get Transaction → Process → End)
`Framework/InitAllSettings.xaml`	Wczytuje Config.xlsx do Dictionary
`Framework/InitAllApplications.xaml`	Otwiera potrzebne aplikacje
`Framework/GetTransactionData.xaml`	Pobiera kolejny element (z Queue lub innego zrodla)
`Framework/Process.xaml`	Logika biznesowa — przetwarzanie jednej transakcji
`Framework/SetTransactionStatus.xaml`	Ustawia status transakcji (Successful/Failed)
`Framework/CloseAllApplications.xaml`	Zamyka aplikacje (graceful close)
`Framework/KillAllProcesses.xaml`	Wymusza zamkniecie procesow (force kill)
`Framework/RetryCurrentTransaction.xaml`	Logika ponawiania transakcji
`Framework/TakeScreenshot.xaml`	Screenshot przy bledzie
`Data/Config.xlsx`	Plik konfiguracyjny (Settings, Constants, Assets)
`Data/Input/`	Dane wejsciowe
`Data/Output/`	Dane wyjsciowe
`Data/Temp/`	Pliki tymczasowe

STANY REFRAMEWORK (MAIN.XAML)

Stan	Co robi	Przejscia
Init	InitAllSettings + InitAllApplications	Sukces → Get Transaction; SE → End
Get Transaction Data	Pobiera kolejny element do przetworzenia	Jest element → Process; Brak → End
Process Transaction	Wykonuje Process.xaml + SetTransactionStatus	Sukces → Get Transaction; BE → Get Transaction; SE → Init
End Process	CloseAllApplications	— (koniec)

Cykl zycia transakcji w REFramework

```
[Init] → ustawienia zaladowane → [Get Transaction]
                                        |
                              Jest transakcja? → NIE → [End Process]
                                        | TAK
                              [Process Transaction]
                                    |    |    |
                               Sukces   BE   SE
                                  |     |    |
                              [Get    [Get  [Init]  ← restart!
                             Trans.]  Trans.] (retry)
```

- **Sukces** → transakcja oznaczona jako Successful, pobierz nastepna
- **Business Exception** → transakcja oznaczona jako Failed (Business), pobierz nastepna (NIE ponawiaj)
- **System Exception** → restart aplikacji, ponow te sama transakcje (do MaxRetryNumber razy)

CONFIG.XLSX — TRZY ARKUSZE

Arkusz	Opis	Kiedy uzywac	Przyklad
Settings	Ustawienia procesu, ktore moga sie zmieniac	Nazwy kolejek, URL-e, konfiguracja retry	OrchestratorQueueName = "MojaKolejka", MaxRetryNumber = 3
Constants	Stale wartosci, ktore sie nie zmieniaja	Timeouty, formaty dat, nazwy arkuszy	Timeout_General = 30000, DateFormat = "dd-MM-yyyy"
Assets	Nazwy assetow z Orchestratora (nie same wartosci!)	Nazwy credentiali, sciezki do aplikacji	CredentialName = "Robot_Login", AppPath_Asset = "Robotivity_Path"

Wazna roznica: W arkuszu Assets nie wpisujemy samych hasel ani wartosci — tylko **nazwy assetow** z Orchestratora. InitAllSettings wczytuje te nazwy, a potem robot uzywa Get Asset / Get Credential z tymi nazwami, aby pobrac rzeczywiste wartosci.

Uwaga techniczna: Config w REFramework jest typu `Dictionary(Of String, Object)` (nie `String, String`!) — dlatego przy odczycie zawsze potrzebny jest `.ToString`: `Config("OrchestratorQueueName").ToString`

ARGUMENTY MAIN.XAML

- `in_OrchestratorQueueName` — nazwa kolejki (mozna nadpisac z Orchestratora)
- `in_OrchestratorQueueFolder` — folder Orchestratora

Co edytuje programista w REFramework?

W praktyce programista edytuje glownie:

- `Process.xaml` — logika biznesowa (przetwarzanie jednej transakcji)
- `InitAllApplications.xaml` — otwieranie aplikacji i logowanie
- `Config.xlsx` — ustawienia, stale, nazwy assetow
- `CloseAllApplications.xaml` — zamykanie aplikacji

Reszta plikow Framework to boilerplate, ktory dziala "out of the box".

ALTERNATYWA BEZ ORCHESTRATORA — WINDOWS CREDENTIAL MANAGER

Jesli nie masz dostepu do Orchestratora, mozesz uzyc **Windows Credential Manager** jako alternatywy:

1. Panel sterowania → Menedzer poswiadczen → Poswiadczenia systemu Windows → Dodaj
2. W UiPath: **Get Secure Credential** zamiast **Get Credential**
   - Target: nazwa poswiadczenia z Credential Manager
   - Output: Username (String), Password (SecureString)

Wazne: Windows Credential Manager dziala tylko lokalnie na danej maszynie. W produkcji zawsze uzywamy Orchestrator Assets.

Modul 4a — Dispatcher

Cwiczenie — Dispatcher do kolejki

Cel: Zaladuj transakcje z Transactions.xlsx do kolejki Orchestratora (hurtowo).

CZEGO SIE NAUCZYSZ

- `Kill Process "excel"` — zapobiega blokadzie pliku przez otwarty Excel
- `Bulk Add Queue Items` — szybki sposob na zaladowanie wielu elementow z DataTable
- FolderPath — sciezka folderu w Orchestratorze (My Workspace)
- Dispatcher jest prosty — cala zlozonosc jest po stronie Performera

Modul 4b — REFramework Performer

Cwiczenie — Performer na bazie REFramework

Cel: Uzywajac szablonu REFramework, stworz Performera, ktory pobiera transakcje z kolejki i przetwarza je w Robotivity.

CO NALEZY SKONFIGUROWAC

1. Config.xlsx

- Settings: `OrchestratorQueueName` = nazwa kolejki
- Settings: `OrchestratorQueueFolder` = folder Orchestratora
- Assets: nazwy assetow (credentials, sciezka do aplikacji)

2. InitAllApplications.xaml

- Get Credential → Username, Password
- Get Asset → sciezka do Robotivity
- Otworz Robotivity i zaloguj sie

3. GetTransactionData.xaml

- Domyslnie: Get Transaction Item z kolejki (juz skonfigurowane w template)

4. Process.xaml

- Wyciagnij dane z `in_TransactionItem.SpecificContent`
- Sprawdz `.IsNumeric`
- Jesli numeryczne → Type Into w Robotivity + Click Accept
- Jesli nie → `Throw New BusinessRuleException("Dane nie sa numeryczne")`

5. CloseAllApplications.xaml

- Kill Process `robotivity`

CZEGO SIE NAUCZYSZ

> REFramework automatycznie obsluguje: retry, logging, status transakcji, restart po SE

> Programista skupia sie na: `Process.xaml` (logika) + `InitAllApplications.xaml` (setup)

> `Throw New BusinessRuleException(...)` — sygnalizuje blad biznesowy (transakcja = Failed)

> Nieobsluzony Exception w Process.xaml → REFramework traktuje jako System Exception → restart Init

> Config.xlsx centralizuje wszystkie ustawienia — latwa zmiana bez edycji workflow

> **Wskazowka:** REFramework to standard produkcyjny. Wiekszosc pracy to edycja Process.xaml i Config.xlsx.

Przydatne

DEBUGOWANIE — DZIEN 3 I 4

Flowchart i State Machine

- **Breakpoint w Flowchart:** Kliknij prawym na FlowStep → Toggle Breakpoint. Robot zatrzyma sie przed wykonaniem tego kroku.
- **Sledzenie Transitions w State Machine:** W trybie Debug (F5) UiPath podswietla aktualny State i Transition.
- **Locals Panel:** W trybie Debug panel Locals pokazuje wartosci wszystkich zmiennych — przydatne do sprawdzenia `TransactionItem`, `SystemException` itp.
- **Slow Step:** Wlacz Slow Step w toolbar — spowalnia wykonanie, widoczne kazde przejscie miedzy stanami.

TYPOWE PROBLEMY I ROZWIAZANIA

Problem	Przyczyna	Rozwiazanie
"Variable not found"	Scope zmiennej za maly	Przesun deklaracje zmiennej wyzej (np. na poziom StateMachine)
NullReferenceException w Dictionary	Dictionary nie zainicjalizowane	Dodaj `New Dictionary(Of String, String)` w Assign
Queue Item pusty	Brak danych w kolejce	Najpierw uruchom Dispatcher
"Queue not found"	Zla nazwa kolejki lub folderu	Sprawdz QueueName i FolderPath w Orchestratorze
Nieskonczona petla w State Machine	Brak warunku wyjscia	Dodaj czyszczenie wyjatkow + poprawne Transitions
Haslo widoczne w logach	Uzycie Text zamiast SecureText	W Type Into uzyj SecureText dla hasla
Plik zablokowany (Excel)	Excel otwarty w tle	Kill Process `excel` przed Read Range
Path Exists zwraca False	Zla sciezka lub PathType	Sprawdz: File vs Folder w PathType